

乒乓球Demo交接文档

1. 项目基本使用教程

- 项目代码

 pingpong_controller.zip
218.98MB



- 项目整体代码

代码块

```
1  .
2  |— package.xml
3  |— pingpong_controller
4  |   |— ball_detector.py           # 球体位置速度提取
5  |   |— calibration_loader.py      # 相机参数提取
6  |   |— data
7  |   |   |— rl_real
8  |   |   |— rl_sim
9  |   |   |— vision_calib          # 相机外参整定数据，包含机械臂关节角与对应
                                     的图像，和相机内参
10 |   |       |— eye_to_hand_sample_848
11 |   |       |   |— images
12 |   |       |   |— realsense_color_camera_info_848.json
13 |   |       |   |— samples.txt
14 |   |       |— vision_refine      # YOLO模型后训练相关数据
15 |   |           |— yolo_image_real # YOLO模型后训练原始数据
16 |   |           |   |— frame_0000_20260501_125843.jpg ...
17 |   |           |— yoloreal-87c9b4bf # YOLO模型后训练数据集
18 |   |           |   |— data.yaml
19 |   |           |   |— images
20 |   |           |   |— labels
21 |   |           |— yoloreal.ndjson
22 |   |— __init__.py
23 |   |— models
24 |   |   |— best_model.zip         # 强化学习策略模型
25 |   |   |— best.pt               # YOLO后训练模型
26 |   |   |— moz1_pd.xml           # 机器人相关模型
27 |   |— outputs
28 |   |   |— pingpong_base_trajectory.png
29 |   |   |— rl_real
```

```

30 | | | | └─ right_arm_joints_test.csv
31 | | | | └─ rl_sim
32 | | | | └─ vision_calib # 相机外参标定结果
33 | | | | └─ eye_hand_result_1280_undistorted.json
34 | | | | └─ eye_hand_result_848_undistorted.json
35 | | | | └─ vision_refine # YOLO后训练结果
36 | | | | └─ predict
37 | | | | | └─ predict_yoloreal_finetune
38 | | | | └─ runs
39 | | | | | └─ yoloreal_finetune
40 | | | | └─ val
41 | | | | | └─ validate_yoloreal_finetune
42 | | └─ pingpong_node.py # 击打乒乓球控制器节点
43 | | └─ __pycache__
44 | | └─ rl_policy.py # 强化学习策略控制器
45 | | └─ safety_limiter.py # Sim2real的安全保障
46 | | └─ tools
47 | | └─ rl_2real # 检测强化学习输出资源包
48 | | | └─ __pycache__
49 | | | | └─ test_arm.cpython-310.pyc
50 | | | └─ test_arm.py
51 | | └─ rl_sim # 强化学习训练资源包
52 | | | └─ config_utils.py
53 | | | └─ config.yaml
54 | | | └─ meshes
55 | | | | └─ base_link.STL ...
56 | | | └─ moz1_pd.xml
57 | | | └─ __pycache__
58 | | | | └─ config_utils.cpython-310.pyc
59 | | | | └─ rl_juggle_env_random.cpython-310.pyc
60 | | | | └─ show_env.cpython-310.pyc
61 | | | | └─ train_juggle_rl_random.cpython-310.pyc
62 | | | | └─ validate_juggle_rl_random.cpython-310.pyc
63 | | | └─ rl_juggle_env_random.py
64 | | | └─ show_env.py
65 | | | └─ Stage
66 | | | └─ training.md
67 | | | └─ train_juggle_rl_random.py
68 | | | └─ validate_juggle_rl_random.py
69 | | └─ vision_calib # 相机外参标定资源包
70 | | | └─ aprilgrid_6x6_25mm_A4.pdf
71 | | | └─ collect_eye_to_hand.py
72 | | | └─ eye_hand_calib.py
73 | | | └─ generate.py
74 | | | └─ __pycache__
75 | | | | └─ collect_eye_to_hand.cpython-310.pyc
76 | | | └─ eye_hand_calib.cpython-310.pyc

```

```

77 |         |   |   |   └─ test_calibration.cpython-310.pyc
78 |         |   |   └─ show_tag.py
79 |         |   └─ test_calibration.py
80 |         └─ vision_refine                                # YOLO模型后训练资源包
81 |             └─ convert_ndjson.py
82 |             └─ predict_fine.py
83 |             └─ __pycache__
84 |             └─ validate_fine.py
85 |             └─ yolo_collector.py
86 |             └─ yolo_fine.py
87 | └─ __pycache__
88 | └─ requirements.txt                                    # conda环境
89 | └─ resource
90 |     └─ pingpong_controller
91 | └─ setup.cfg
92 | └─ setup.py
93 | └─ test
94

```

- 部署相关代码包

部署代码

```

1  cd /home/jacky/pingpong_ws
2  cd src/pingpong_controller
3  pip install -r requirements.txt
4  conda activate pingpong
5  python -m colcon build --packages-select mc_core_interface --symlink-install
   --cmake-args -Wno-dev -DPython3_EXECUTABLE=$(which python) --event-
handlers console_direct+

```

- Realsense D455 相机驱动启动（本部分所有模块都需要依赖相机驱动的启动）：

Realsense 驱动启动

```

1  ros2 launch realsense2_camera rs_launch.py \
2    enable_depth:=false \
3    enable_color:=true \
4    rgb_camera.color_profile:=848x480x60 \
5    rgb_camera.color_format:=RGB8 \
6    pointcloud.enable:=false \
7    align_depth.enable:=false

```

- 启动节点

```

启动节点/home/jacky/pingpong_ws
2  source install/setup.bash
3  ros2 run pingpong_controller pingpong_node --ros-args \
4    -p use_direct_realsense:=false \
5    -p image_topic:=/camera/camera/color/image_raw/compressed \
6    -p camera_info_topic:=/camera/camera/color/camera_info \
7    -p
   calibration_json_path:=/home/jacky/pingpong_ws/src/pingpong_controller/pingpong
   _controller/outputs/vision_calib/eye_hand_result_848_undistorted.json \
8    -p joint_state_topic:=/joint_states \
9    -p command_topic:=/mx_mix_command \
10   -p control_rate_hz:=200.0 \
11   -p rl_policy_dt:=0.005 \
12   -p
   robot_xml_path:=/home/jacky/pingpong_ws/src/pingpong_controller/pingpong_contro
   ller/models/moz1_pd.xml \
13   -p
   rl_model_path:=/home/jacky/pingpong_ws/src/pingpong_controller/pingpong_control
   ler/models/best_model.zip
14
15  # 关键topic检查
16  ros2 topic echo /joint_states --once
17  ros2 topic echo /pingpong/ball_state --once
18  ros2 topic echo /pingpong/rl_joint_cmd_state --once
19  ros2 topic echo /mx_mix_command --once

```

2. 相机外参标定

本节主要针对在实际运行时，视觉感知模块输出的为在相机光心坐标系下的球体位置和速度，为减少 Sim2Real Gap，在训练时主要获取球体在机器人 Base 坐标系下的特权信息，因此需要做相机光心坐标系与机器人 Base 系之间的外参标定。方法主要参考相关的眼在手外且固定的标定方法：

<https://zhuanlan.zhihu.com/p/458993915>

相关脚本及流程如下：

- **生成 AprilGrid 标定板（可选）**：生成针对不同纸张大小的标定纸，打印出来后固定至左臂末端执行器（为例）

标定板生成

```

1  python3 pingpong_controller/pingpong_controller/tools/vision_calib/generate.py
   \
2    --rows 6 \
3    --cols 6 \
4    --tag-size-mm 25 \
5    --spacing 0.3 \

```

```
6 --family tag36h11 \  
7 --paper A4 \  
8 --output  
pingpong_controller/pingpong_controller/tools/vision_calib/aprilgrid_6x6_25mm_A  
4.pdf
```

- **AprilTag实时检测**：为了在收集数据时能够实时观测是否有足够的tag，即光照、距离、清晰度等外界条件是否可行，可运行以下程序，退出按窗口里面的 **q** 键

Tag实时检测

```
1 python3 pingpong_controller/pingpong_controller/tools/vision_calib/show_tag.py  
\  
2 --image-topic /camera/camera/color/image_raw \  
3 --tag-family tag36h11 \  
4 --max-id 36
```

- **eye-to-hand标定样本采集**：运行采集脚本，订阅相机 RGB 图像、信息以及机器人本体的关节角信息，按空格保存当前图像+关节角对应信息，按 **q** 键退出，

样本采集

```
1 python3  
pingpong_controller/pingpong_controller/tools/vision_calib/collect_eye_to_hand.  
py
```

注：采集信息将会保存到data/vision_calib/eye_to_hand_samples_20260502_153000/，里面包含图像、样本的文本信息以及对应采集过程中相机的内参信息。

注：采集建议：至少20-30长，标定板在图像中不同位置、角度、距离都要覆盖，且每张图需要至少能看到8个以上的tag，避免强反光、运动模糊、遮挡，机械臂姿态需要有变化，不要只在一个小范围内运动

- **外参标定**：利用链接方法进行外参标定

外参标定

```
1 python3  
pingpong_controller/pingpong_controller/tools/vision_calib/eye_hand_calib.py \  
2 --undistort \  
3 --write-undistorted-intrinsics {去畸变内参输出路径} \  
4 --intrinsics {原始内参路径} \  
5 --samples {sample.txt路径} \  
6 --image-dir {image 文件夹路径} \  
7 --urdf /path/to/moz1.urdf \  

```

```

8    --tag-family tag36h11 \
9    --tag-rows 6 \
10   --tag-cols 6 \
11   --tag-size 0.025 \
12   --tag-spacing 0.3 \
13   --ee-frame left07 \
14   --head-frame head23 \
15   --base-frame base_link \
16   --method PARK \
17   --min-tags 8 \
18   --max-reproj-error 10 \
19   --corner-order 0123 \
20   --output
    pingpong_controller/pingpong_controller/outputs/vision_calib/eye_hand_result_84
    8_undistorted.json \
21   --debug-dir
    pingpong_controller/pingpong_controller/outputs/vision_calib/debug_vis_848_undi
    storted

```

输出重点观察 `Done. xx used, yy skipped. Mean reprojection error: ...`

`Hand-eye residual: ... mm mean, ... mm max` 以及 `Result: ...`，来观测对应的重投影误差，以及最后标定的时候的手眼误差

- **标定结果检查：**检查 JSON 能否加载，内参矩阵维度是否正确，以及对应的外参矩阵是否合法，单位换算正确

标定结果检查

```

1  python3
    pingpong_controller/pingpong_controller/tools/vision_calib/test_calibration.py
    \
2  {标定外参路径}

```

3. 检测模型后训练

本节主要针对 YOLO 模型在本任务的微调教程进行介绍（本部分不依赖 ROS 包结构，为直接 PYTHON 脚本），开始前请确保已启动相机驱动和 Conda 环境

- **采集真实图片：**运行 RealSense 图像采集脚本，操作方式为 `“空格键：保存当前帧”` `”q 键：退出“`

微调采集图片脚本

```

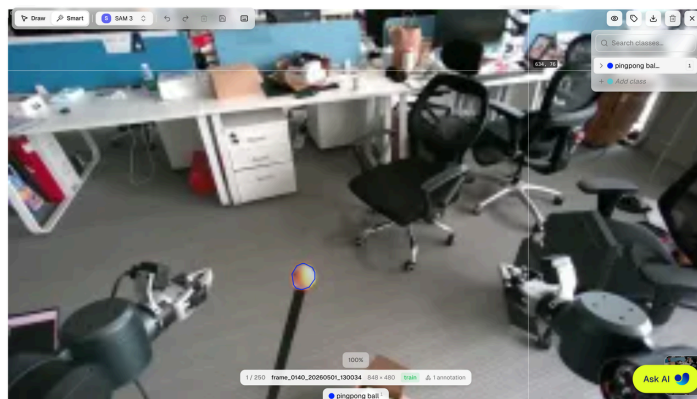
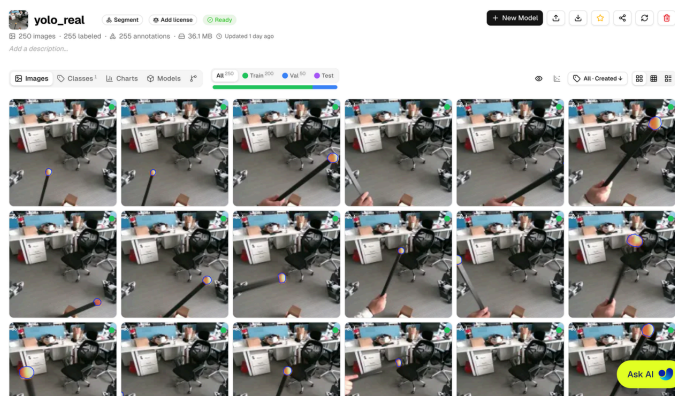
1  python3

```

pingpong_controller/pingpong_controller/tools/vision_refine/yolo_collector.py

注：图像为保存至 `data/vision_refine/yolo_image_pingpong/20260502_153000/`

- **数据标注与下载：**将采集数据上传至网站：<https://platform.ultralytics.com/jinke-yao/datasets/yoloreal>，并通过创建新数据后（注意选择 YOLO Segmentation），利用 SAM3 辅助数据标注，并最终下载 NDJSON 数据文件至 `data/vision_refine/*` 作为后续微调数据集



- **本地数据集转换：**将导出的标注文件转换为 YOLO 格式数据集目录

数据集格式转换

```
1 python3
   pingpong_controller/pingpong_controller/tools/vision_refine/convert_ndjson.py \
2   {ndjson文件地址} \
3   {optional: --output-dir data/vision_refine/yoloreal_dataset}
```

转换后重点关注目标文件夹是否包含 `data.yaml` 文件

- **微调YOLO模型：**运行训练脚本

微调模型

```
1 python3
   pingpong_controller/pingpong_controller/tools/vision_refine/yolo_fine.py \
2   {数据集中 data.yaml 文件路径} \
3   --base-model yolov11s-seg.pt \
4   --epochs 50 \
5   --imgsz 848 \
6   --batch 8 \
7   --device 0 \
8   --name yoloreal_finetune
```

训练输出路径：`outputs/vision_refine/runs/yoloreal_finetune/`

- **验证微调模型指标：**训练完成后，用 `best.pt` 跑验证，会打印 `mAP50-95`，`mAP50`，`Precision Recall`，`segmentation mAP`

验证指标

```
1 python3
  pingpong_controller/pingpong_controller/tools/vision_refine/validate_fine.py \
2   {best.pt 路径} \
3   {data.yaml 路径} \
4   --name validate_yoloreal_finetune
```

输出默认会在 `outputs/vision_refine/val/`

- **预测质检：**用训练好的模型对真实图片文件夹做可视化预测

可视化分割

```
1 python3
  pingpong_controller/pingpong_controller/tools/vision_refine/predict_fine.py \
2   {best.pt 路径} \
3   {图片路径} \
4   --imgsz 848 \
5   --conf 0.25 \
6   --device 0 \
7   --name predict_yoloreal_finetune
```

输出图片会在 `outputs/vision_refine/predict/predict_yoloreal_finetune/`

- **模型部署：**若训练模型指标达到预期，请复制到 `/model` 路径下，并命名为 `best.pt` 以方便对应程序加载

4. 强化学习策略训练

本节主要针对强化学习训练包进行教程介绍，在进入对应的目录之后，这个目录是独立 RL 仿真训练工具，不需要 `colcon build`，也不作为 ROS 包运行。目录里相关主要文件：

目录主要文件

1	<code>rl_juggle_env_random.py</code>	强化学习环境
2	<code>train_juggle_rl_random.py</code>	训练 PPO
3	<code>validate_juggle_rl_random.py</code>	验证/录视频/画轨迹
4	<code>show_env.py</code>	打开 MuJoCo 看环境
5	<code>config.yaml</code>	默认训练/验证配置
6	<code>training.md</code>	课程学习每阶段命令
7	<code>moz1_pd.xml</code>	MuJoCo 机器人 XML

- 首先检查环境是否正常

环境检查

```
1 python3 show_env.py
2 python3 show_env.py \
3     --xml
   /home/jacky/pingpong_ws/src/pingpong_controller/pingpong_controller/tools/rl_si
   m/moz1_pd.xml
```

后续相关训练、验证、测试等部分可直接参考其中的 `train.md` 文件